

# PROFESSIONAL RECORD (PD)

|                   |                                  |
|-------------------|----------------------------------|
| <i>Birth Name</i> | ▶ LESIEUR                        |
| <i>Usage name</i> | ▶                                |
| <i>First Name</i> | ▶ DYLAN                          |
| <i>Address</i>    | ▶ CALLE GALERA 39, MADRID, 28042 |

## Intended professional designation

WEB AND MOBILE WEB DEVELOPER

### HOW TO GET THERE:

- ☒ Learning Path
- ☐ Validation of Prior Experiential Learning (VAE)

## Presentation of the application

The Professional Record (PA) is one element of the designation validation system. **This permit is issued by the Ministry of Employment.**

The PD belongs to the candidate. He keeps it, updates it during his course and must present it **at each examination session.**

To write the DP, the candidate can be helped by a trainer or by a VAE coach.

It is consulted by the jury at the time of the examination session.

### To make its decision, the jury has:

1. the results of the professional situation, supplemented, if necessary, by the professional questionnaire or the professional interview or the technical interview or the questioning based on productions.
2. the **Professional Record** (PD) in which the candidate has recorded evidence of his or her professional practice
3. the results of evaluations taken during the course of the course when the candidate being evaluated comes from a training course
4. the final interview (as part of the title session).

*[Decree of 22 December 2015, relating to the conditions for the issuance of professional titles of the Ministry of Employment]*

### This file includes:

- ▶ for each typical activity of the title in question, one to three examples of professional practice;
- ▶ a table to be filled in if the candidate wishes to bring to the attention of the jury the possession of a title, a diploma, a certificate of professional qualification (CQP) or training certificates;
- ▶ a sworn statement to be completed and signed;
- ▶ documents illustrating the candidate's professional practice (optional)
- ▶ annexes, if necessary.

*To complete this file, the candidate has a website with free access on the site.*



<http://travail-emploi.gouv.fr/titres-professionnels>

## Contents

### Examples of professional practice

|                                                                                          |              |
|------------------------------------------------------------------------------------------|--------------|
| <b>Develop the front-end part of a secure web or mobile web application</b>              | <b>p. 5</b>  |
| ▶ Install and configure your work environment according to the web or mobile web project | p. 5         |
| ▶ Mockup web or mobile web user interfaces                                               | p. 8         |
| ▶ Realize static web or mobile web user interfaces                                       | p. 11        |
| <b>Develop the back-end part of a secure web or mobile web application</b>               | <b>p. 15</b> |
| ▶ Set up a relational database                                                           | p. 16        |
| ▶ Develop SQL and NoSQL data access components                                           | p. 20        |
| ▶ Develop server-side business components                                                | p. 25        |
| <b>Titles, diplomas, CQP, training certificates</b> <i>(optional)</i>                    | p. 30        |
| <b>Declaration of honour</b>                                                             | p. 31        |
| <b>Documents illustrating professional practice</b> <i>(optional)</i>                    | p. 32        |
| <b>Appendices</b> <i>(If provided for in the RC)</i>                                     | p. 33        |

# **EXAMPLES OF PRACTICE PROFESSIONAL**

## Typical activity 1

Develop the front-end part of a secure web or mobile web application

**Example 1** ▶ *Install and configure your work environment according to the web or mobile web project*

### 1. Describe the tasks or operations you performed, and under what conditions:

*When I started the Vite Gourmand project, the very first step was to set up a development environment. This was one of the prerequisites for working efficiently and ensuring that the application could be relaunched in the same way on any machine, thus avoiding the "it works for me but not for others" effect. The goal of the project was to create a full-stack web application for a gourmet caterer.*

#### **Analysis of the technical environment and reading of specifications**

*Before installing anything, I mapped (see Fig. 15) All technical components:*

- front-end (React, Vite, TypeScript)
- back-end (NestJS, Prisma)
- two databases (PostgreSQL [Supabase], MongoDB [Atlas])
- des services tiers (SMTP, API Unsplash, IA).

*This reading from the mindmap allowed me to predict the necessary environment variables before writing the first line of code.*

#### **Installing the developer tools**

*On Linux Ubuntu (virtual machine), I installed Visual Studio Code with extensions like: TypeScript, ESLint, Prettier, Tailwind CSS IntelliSense, Node.js LTS 22, PostgreSQL clients, and Prisma command-line tools.*

#### **Containerization with Docker**

*I wrote a docker-compose.yml file defining a single service based on Node.js 22 Alpine, running both servers in the same container, with named volumes to persist node\_modules as well as healthchecks at startup. A Makefile orchestrates all common operations with the following targets: all, bootstrap, local, deploy, test, secrets, stop, shell, fclean, help (see Fig. 8).*

*42madrid@fundaciontelefonica.com*

#### **Secrets management with Bitwarden CLI**

*A script based on the Bitwarden CLI automatically retrieves secrets (connection strings, JWT keys, SMTP IDs, API tokens) from the vault when containers are launched and injects them as environment variables. The .gitignore file is configured to exclude generated files and potential leaks of sensitive information.*

## **Versioning and continuous integration**

*I initialized the Git repository, set up a repository on GitHub, and set up GitHub Actions workflows to automatically check the TypeScript build, unit tests, as well as SonarCloud (a QA platform) on each push (see Fig. 11) which analyzes security vulnerabilities, maintainability, and best practices. All of this can be seen from Fig. 14. New jobs have been added to each iteration to automatically check the health of the branches.*

## **Difficulties encountered**

*Connecting to Supabase from the container required a specific parameterization: the `pgbouncer=true` and `connection_limit=1` options, identified by the analysis of the error logs (see Fig. 20).*

## **Environmental Verification**

*Once the environment was finally up and running, I checked the front-end startup, NestJS API response with accessible Swagger documentation, error-free TypeScript compilation, and effective connection to both databases via real endpoints.*

## **Infrastructure refactoring**

*As the development progressed, I refactored the organization of Docker services into a dedicated tree (see Fig. 9 and Fig. 17)*

*, clearly separating responsibilities between application services, databases and point tools (Bitwarden CLI, Fly CLI). This separation eliminated configuration conflicts and made each service independent and replaceable. Although the monolithic approach is often recommended for a project of this size, the incompatibilities between the different workstations motivated the choice of this infrastructure.*

*This organization is more demanding to maintain, but it guarantees total reproducibility of the environment.*

*As for the Makefile, it has been structured in modules in a `mk_extensions/` directory, each file grouping the targets by domain. This modularity allows new targets to be added without weighing down the main file, which acts as executable documentation.*

## **Continuous Integration Chain Fixes**

*The GitHub Actions configuration required several fixes to be up and running. A TypeScript configuration issue generated incorrect `rootDir` paths and prevented CI compilation. This was resolved by adjusting the paths in `tsconfig.json`. In fact, Jest required the `--localstorage-file` option to isolate storage between tests in a CI environment, thus avoiding interference between parallel test suites.*

## **2. Specify the means used:**

# PROFESSIONAL RECORD (PD)

**Visual Studio Code** : extensions TypeScript, ESLint, Prettier, Tailwind CSS IntelliSense.

**Node.js 22 LTS** : Runtime environment for NestJS and Vite; Docker and Docker Compose: containerization on Node.js 22 Alpine image.

**Makefile** : unified interface (start, stop, logs, cleanup); Bitwarden CLI: Secure injection of secrets at startup.

**Git and GitHub** : Version control, remote repository, GitHub Actions for TypeScript CI.

**Prisma CLI** : schema management and client generation; Supabase (PostgreSQL) and MongoDB Atlas: Cloud databases.

## 3. Who have you worked with?

*This project was developed in complete autonomy. The resolution of technical problems was based on the official documentation (Docker, NestJS, Prisma, Vite), specialized forums (Stack Overflow, Discord NestJS and Prisma) and the error logs of the tools. No external collaborators were involved in the code.*

## 4. Background

Name of the company, organization or association ▶

*Independent training — Vite Gourmand Project*

Construction site, workshop, service

▶ *Full-stack development in autonomy*

Fiscal Period

▶ **Fro 01/01/2026 to 15/02/2026 m**

## 5. Additional information (optional)

*Docker containerization ensures the repeatability of the environment on any machine. The Makefile centralizes all available operations. Secrets management through the Bitwarden CLI ensures that no sensitive data is ever versioned.*

## Typical activity 1

Develop the front-end part of a secure web or mobile web application

Example 2 ▶ Mock up web or mobile web user interfaces

### 1. Describe the tasks or operations you performed, and under what conditions:

*In this same project, I was responsible for designing and formatting the interfaces of the application, which was aimed at two distinct audiences: customers and professionals.*

#### **User Needs Analysis**

*Before composing my first UI elements, I analyzed the specific needs of each type of user. For the general public, the interface had to evoke a gastronomic and premium environment with great ease of navigation.*

*For the professional public, it had to be practical as a priority, in the sense that professionals will want to manipulate data and interact quickly with customers.*

*I translated these requirements into concrete visual identity choices: a palette centered on deep burgundy, golden champagne, olive green and cream, and two typographic families. For the font, I chose Inter for readability at all sizes, and Playfair Display for the titles in the public storefront.*

#### **Sketching and page structure**

*I then worked freehand to lay down the general structure of the pages. These sketches I made represented later the home page, the organization of the menu catalog, and the navigation between the storefront and the admin portal. These low-fidelity sketches allowed me to validate the navigation logic before investing time in the visual details (see Fig. 1 and Fig. 2).*

*I knew Figma, but after several days of learning all his tools, I still wasn't moving forward on the project. I needed to simplify the process. That's why I chose to work with Excalidraw. Simple and powerful, it's a versatile tool that can be easily used to create low-fidelity mockups.*

#### **Mobile route design**

*I paid particular attention to the mobile paths: the order of the sections on a vertical screen, the position of the action buttons within thumb's reach, the transformation of the navigation menu into a full-screen panel.*

#### **CSS token system**

*The project's decision was to build the entire design on a CSS token system before designing any interface. I organized this system into three graphic charter files (see Fig. 16) :*

**The first** concentrating all color tokens (ten hue levels per semantic color, background colors slightly tinged with purple, text colors expressed in opacities).

**the second** grouping together construction tokens (eight-level typographic scale, four font weights,



# PROFESSIONAL RECORD (PD)

*thirteen spacing values, six border radius levels, five drop shadow intensities).*

***the third** Managing behavior tokens (eight z-index levels, three transition speeds, standardized component dimensions, layout tokens).*

## **Design of the page Legal notice and cookies**

*CNIL compliance required a page dedicated to transparency on cookies.*

*A legible table presenting the seven cookies used (Name, Use, Duration, Type),  
coloured badges (Essential, Functional, Third Party),*

*a "How to change your mind" block explaining how to revoke consent,*

*a "Your GDPR rights" block listing the rights (access, rectification, deletion, opposition, portability) with a direct link to the CNIL form.*

*The goal: for a non-technical user to understand in less than a minute what is stored, why, and how to delete it.*

## **2. Specify the means used:**

**Paper sketches** : low-fidelity structuring of routes and layouts before any implementation.

Excalidraw: more dynamic low-fidelity structuring and relationship building.

**Native CSS variables** : A three-file system of brand guidelines covering colors, typography, spacing, shadows, transitions, and component dimensions.

**Tailwind CSS 4 + CVA** : Utility Framework with Custom Branded Palette

**Radix UI** : accessible primitives (ARIA, focus, keyboard) stylized with the tokens of the design system;

Lucide **React** : SVG icons imported individually; @axe-core/react: accessibility audit in development.

**DevTools Chrome** : CSS variable inspection, mobile simulation, contrast checking; Radix UI, Tailwind CSS, CVA, and MDN documentation for advanced CSS functions.

## **3. Who have you worked with?**

# PROFESSIONAL RECORD (PD)

*This project was carried out in total autonomy, leading me to simultaneously occupy the roles of designer and developer. Each design choice was immediately evaluated in terms of its technical feasibility and maintainability. The design was done directly in the browser, with the DevTools acting as a visual validation tool. Community resources (Radix UI documentation, Tailwind forums, front-end Discord) have occasionally served as sources of validation.*

## 4. Background

Name of the company, organization or association ▶ *Independent training — Vite Gourmand Project*

Construction site, workshop, service  
Fiscal Period ▶ *Full-stack development in autonomy*  
▶ *From 01/01/2026 to 15/02/2026*

## 5. Additional information (optional)

*The CSS token system proved its effectiveness when adding the dashboard: all new components were built in a matter of hours without creating new tokens. A value change propagates instantly across the entire interface. This is concrete proof of a design system maintained by convention rather than by copy.*

## Typical activity 1

Develop the front-end part of a secure web or mobile web application

**Example 3** ➤ *Realize static user interfaces and develop the dynamic part of web or mobile web user interfaces*

### 1. Describe the tasks or operations you performed, and under what conditions:

*As part of this project, I created the entire static user interfaces of the web application, covering five main public pages: the home page, the menu catalog, the contact page, the order page and the legal pages.*

#### **HTML Semantic Structure**

*I paid particular attention to the semantic structure of each page:*

- `<nav>` tag for the navigation bar;
- `<header>` for the header;
- `<main>` for the main content;
- `<section>` for each thematic block;
- `<item>` for each menu or testimonial card;
- and `<footer>` for the footer.

- ...

*The root HTML file explicitly declares the French language (`lang="fr"`), the UTF-8 character set, and the viewport tag essential for mobile rendering. The titles follow a strict hierarchy: a single `<h1>` per page, `<h2>` for section titles, `<h3>` for map titles.*

#### **Formatting with Tailwind CSS**

*For the formatting, I used Tailwind CSS 4 in combination with the custom CSS variable system defined during the mock-up phase. The layout is based on CSS and Flexbox grids via the Tailwind utilities, with a systematic mobile-first approach: each grid is first designed as a single column on mobile, then enriched in stages for larger screens.*

#### **Media Optimization**

*The images come from Unsplash, which optimizes them on demand via URL parameters (format, dimensions, quality). Lucide React icons are imported as a unit so that the Quick bundler only includes those actually used.*

## Accessibility

*In an increasingly inclusive world, accessibility by design is an essential choice that substantially improves the quality of a web page. That's why complex interactive components (modals, drop-down menus, accordions) are built on Radix UI, which natively handles ARIA attributes, focus management, and keyboard navigation. As for the focus state, it is visible and set globally for all interactive elements. In addition, html semantics, if respected, is a fundamental for web accessibility. For example, screen readers like NVDA or VoiceOver mainly navigate through the semantic structure. A `<nav>` announces "navigation area", `<main>` allows you to jump directly to the main content, etc.*

*Concretely in the project: if you have a `<article>` for each menu card, a screen reader can list all the articles on the page and navigate between them directly. If we had put `<div>` everywhere instead, this navigation would be impossible semantic unless we used the aria attributes.*

## Tests and verifications

*Once each interface was made, I carried out a systematic check: display test on the main resolutions via Chrome DevTools, verification of keyboard navigation on each page, and Lighthouse audit to evaluate performance, accessibility, SEO, and best practices. These tests guided the construction of the website.*

## UI Security and CNIL Compliance

*Front-end security was addressed through 11 issues listed in the code space.*

*The main flaw was the JWT stored in localStorage, accessible from JavaScript and therefore exposed to XSS attacks. The strategy to fix this outcome was to migrate to an HttpOnly + Secure cookie policy making the token inaccessible to client code. To enhance security, a CSRF mechanism via the X-CSRF-Token header has also been implemented. Thus, the server issues the token in a readable cookie that the client reads and returns in each mutant request.*

## Quality and SEO

*Six main routes were audited with Lighthouse (Performance, Accessibility, Best Practices, SEO), scoring above 90 on each (see Fig. 12 and Fig. 13). For natural referencing, I have set up a file robots.txt allowing the indexing of the entire site (Allow: /), a sitemap.xml listing the six searchable URLs (see Fig. 18).*

*According to <https://developers.google.com/docs/crawling-indexing/robots/intro>. It's a text file that we will put at the root of our site to communicate to search engine robots like googlebot. You have to think of this file as a directional sign that robots would read. For each index to be made, it reads the instructions to be respected. In our case, we simply blocked pages for indexing and improved SEO. Without this file, Google indexes and interprets what it wants. Instead, we can decide to give him the map of our website to visit. To prevent admin pages from appearing in natural referencing. You can simply Disallow.*

*What I learned above all is that this robots.txt should not be confused with security. It's just a convention. A robot with bad intentions can simply ignore it. Admin routes remain accessible to anyone who enters the entire URL. The real protection is the use of JWT tokens in NestJS.*

*Concretely, someone who searches for "caterer paris", Google will consult its database to display a link to*

# PROFESSIONAL RECORD (PD)

other websites that will probably not know fast-greedy because they are not based in Paris.

## 2. Specify the means used:

**Visual Studio Code** : extensions TypeScript, ESLint, Prettier, Tailwind CSS IntelliSense ; Vite 7 : serveur de développement HMR, bundler Rollup en production.

**HTML5 semantics**: structure tags (nav, header, main, section, article, footer), title hierarchy, lang and viewport attributes.

**Tailwind CSS 4 + native CSS variables**: mobile-first utility framework with color tokens, typography, and spacing in three brand identity files; CSS clamp() for smooth typography.

**Radix UI (accessible ARIA/focus/keyboard components)**, Lucide React (single SVG icons), @axe-core/react (accessibility audit in development).

**Lighthouse + DevTools Chrome**: performance, accessibility, SEO; Unsplash CDN: Optimized images on demand; Git and GitHub: versioning.

## 3. Who have you worked with?

Project conducted in complete autonomy. Alone in the roles of designer, integrator and front-end developer, I validated my decisions via the automated audit tools (Lighthouse, axe-core), the DevTools for responsive behavior, and the official W3C documentation. Community forums (Tailwind Discord, Radix UI documentation) have been used as a reference for cases not covered by the documentation.

## 4. Background

Name of the company, organization or

Independent training — Vite Gourmand Project

# PROFESSIONAL RECORD (PD)

## association ▶

|                                         |                                      |
|-----------------------------------------|--------------------------------------|
| Construction site,<br>workshop, service | ▶ Full-stack development in autonomy |
| Fiscal Period                           | ▶ From 01/01/2026 to 15/02/2026      |

## 5. Additional information (optional)

*Eco-design: lazy loading of images and unit import of icons reduce the data downloaded. Forms security: client-side validation (native HTML, required/maxlength) coupled with server-side validation via class-validator for complete coverage.*

## Typical activity 2

Develop the back-end part of a secure web or mobile web application

Example 1 ► *Setting up a relational database*

### 1. Describe the tasks or operations you performed, and under what conditions:

In the context of this project, one of the most structuring tasks of the back-end was the design and implementation of the relational database. The app actually covers a wide range of jobs — order management, customer loyalty, menus with allergens, internal messaging, support, human resources, and so on. The database had to be designed from the outset as a coherent, scalable and secure system.

#### Requirements gathering and conceptual analysis

Before defining any table, I listed all the necessary business entities. This analysis work has enabled me to identify fortysix Distinct data models, grouped into major thematic families such as (users and authentication, business management, menus and dishes, orders and deliveries, etc.). The diagram being much too big for a simple screenshot in word (see Fig. 5); So I had to omit this part. For each entity, I defined the necessary attributes, appropriate data types, integrity constraints, and relationships with the other entities.

#### Defining the physical schema with Prisma

Many MNOs exist. I finally chose Prisma which in the future inspired me to create another project called Prismatic. It is an ORM used as a model, a typed client generator and migration managers. That is, thanks to Prisma, we will describe the tables in TypeScript in a single file called schema.prisma. Here we will never write SQL. Prisma reads this file and understands the structure of the database. From this schema, Prisma automatically generates TypeScript code that can be used directly. When you change the schema, such as adding a column or renaming a table. Prisma generates a migration file. Each migration file contains the exact SQL that changed the database. We can replay the history. Go back, or apply the same changes on another server. Without this ORM we would have had to do all three by hand: write the SQL for creating the tables, write the SQL queries in the code, manage the migration scripts myself.

#### Designing relationships between tables

The application is based on three types of relationships. One-to-many relationships are the most common: a user can place multiple orders, a loyalty account can have many transactions (see Fig. 3).

My approach was deliberately top-down: I didn't start creating tables from the start, but I first mapped all the business processes (ordering, loyalty, menu management, human resources, messaging, support). This census brought out the features and their links before any Prisma diagram was written. This preliminary analysis phase has saved me from the late restructuring that costs me a lot in terms of migration.

# PROFESSIONAL RECORD (PD)

For many-to-many relationships—allergen tags on dishes, roles on users, promotions on orders—I systematically created explicit junction tables rather than relying on Prisma's implicit relationships. This choice allows the relationship to be enriched with additional business attributes (date of application of a promotion, source of assignment of a role) without restructuring the schema. The rarer one-to-one relationships have been used to isolate sensitive data: the loyalty profile is separated from the user account so that it can be deleted independently in the event of a GDPR request.

On the issue of normalization, I applied a clear rule: everything that must remain consistent and searchable with transactional constraints goes into PostgreSQL in normalized form; anything that is a frozen snapshot of a past state (order snapshot, aggregated statistics, audit log) goes into MongoDB in denormalized form. This separation avoids heavy joins on historical data while ensuring the integrity of the live data.

Finally, the use of Prisma Migrate made it possible to evolve the schema in an iterative and traceable way. Each change — adding a field, renaming a relationship, introducing a partial index — resulted in a versioned migration file. This allowed me to faithfully replay the history of the schema on a new environment and to go back in case of error, without ever directly touching the base.

## Setting up indexes

Thirty-four indexes were defined on frequently accessed columns: order, review, and promotion status columns for filters; date columns for chronological sorting; most commonly used foreign keys. Some indexes are partial. The index on active newsletter subscribers only covers records that is\_active is true. This reduces the size of the index and improves the performance of common queries.

Indexes can sometimes be confused with other mechanisms: it is important to distinguish between a SQL index (an internal structure in the database that speeds up server-side queries) and an indexing by a search engine such as Google (public referencing of a page on the web). The two have no relation and do not act on the same performance.

## Data security and privacy

Data security was addressed on several levels. First, passwords are hashed with bcrypt at twelve rounds of salting. The password policy has been reinforced (minimum length of eight characters, uppercase, lowercase, number), applied when changing passwords. In addition, JWT tokens have a lifespan of fifteen minutes. Password reset tokens are random strings of thirty-two bytes, stored with an expiration of one hour and marked as used after their first consumption.

Initially, it cost me a lot to understand how encryption works. In fact, for years and still today, encryption is surely one of the biggest challenges developers have ever faced, much like Pandora's box. Here's how I figured it out, from riskiest to least risky.

The initial problem is that if the developers stored the passwords in plain text, the person who accesses the database directly obtains all the passwords. For a long time, simple hashing has been used in the community: MD5 and SHA-256.

If we use SHA256("test123!") which would have an output "xxxxxxxxxx" in hexadecimal. An attacker can simply create a hash table, precompute them in advance, and retrieve passwords in milliseconds. This is called a rainbow table. This is even more obvious if multiple people have the same hash. The model is then too predictable. Salting is a unique random value generated for each user, added to the password before chopping.



This generates totally different hashes for the password. The rainbow table becomes useless and the attacker should have generated billions and billions of values since he would have to calculate for each salt. However, salt is not a secret and it is easy to find the password since salt is contained in the hash. Even with a salt, it's still too fast to find the password since a GPU can test billions and billions of combinations per second.

Bcrypt solves this with a cost factor that makes the computation intentionally slow (see Fig. 19). In the project we used  $2^{12} = 4096$  iterations of hashing per password. ~300ms for the project. According to OWASP, it is advisable to put at least 10 or 12 for more reinforcement. If you choose 14, the time would be 4 times slower per password than 12. That's about 1200 ms. For a user who connects it's 300ms. For a striker, it's 9.5 years of calculation.

$1,000,000,000 * 0.3 \text{ sec}$

$= 3,000,000 \text{ seconds}$

$= 9.5 \text{ years (per CPU core)}$

The salt is stored in the base (visible if the database is stolen). Pepper is a secret on the waiter's side, never as a base.

Hash = bcrypt(mot de passe + PEPPERSECRET, SALTROUND)

In the project, peppering is not implemented. password.service.ts directly calls bcrypt.hash(password, SALT\_ROUNDS) without adding any server secrets. It's assumed and it's not critical since bcrypt + salt is already very solid. But it is a real layer of defence in depth that is missing.

This means that if the attacker only steals the database, he cannot crack the hashes without knowing the pepper stored in the server's environment variables.

Note: a brute force solution would be to increase the computing capacity of the hardware components. If an attacker were to attack, he would need GPUs. An RTX 4090 on bcrypt would do about 12,000 hashes/second.

$1,000,000,000 / 12,000 = 83,333 \text{ seconds} \approx 23 \text{ hours.}$

With 10 GPUs, it would be about 2 hours.

With a GPU farm, probably a few minutes.

This is not sustainable. Today they use dictionaries. A password like 123456 would take a few ms to be discovered. A password with Test123!, probably a few seconds. A more complex password like this "@#~#@1/2|@~#~1/2#~{-1/2{" would take much longer.

In addition, there is a password + rate limiting NestJS policy with, for example, 20 requests/sec max. + password retrieved by email.

### GDPR Compliance

When a user clicks on "I accept", we don't just display a banner and forget. Each consent is recorded in the database with three pieces of information. What was accepted, when, and since she IP address. If a

# PROFESSIONAL RECORD (PD)

judge or the CNIL asks one day if a user has consented. The answer is traceable. When someone requests the deletion of their account, we don't delete the line in the database. We do what we call anonymization. His email becomes a generic value, the attributes related to his identity are erased. Its orders are kept: a company has the legal obligation to keep its accounting data for several years, even if the customer leaves. The deletion request itself is archived, along with the date it was processed and the ID of the admin who executed it.

## Setting up test data

To enable development and testing under realistic conditions, I created a complete dataset divided into forty-five SQL files: twenty users representing all roles, thirty-five menus, twenty orders with various statuses, loyalty accounts, reviews and support tickets. Test users' passwords are hashed with bcrypt, just like in production. In addition, a global validation pipe (class-validator, class-transformer) inspects each incoming request, rejects undeclared properties, and returns a response listing each invalid field with the reason for rejection.

## 2. Specify the means used:

# PROFESSIONAL RECORD (PD)

**PostgreSQL (Supabase cloud) + Prisma ORM** : schema modeling, typed TypeScript client generation, migration management via Prisma CLI; @prisma/adapters adapter with pg driver Node.js.

**Supabase** : Two separate connection URLs — PgBouncer transactional pooler (port 6543) for the application, direct connection (port 5432) for migrations.

**bcrypt (12 rounds)**: hashing passwords; class-validator + class-transformer: validating and transforming incoming data.

**Bitwarden CLI**: Secure injection of connection strings; Docker: reproducibility of the environment; Git and GitHub: Versioning of seed's schema and SQL files.

**Official documentation**: Prisma (relationships, migrations, adapters), PostgreSQL (types, partial indexes), bcrypt (cost factor), Supabase (PgBouncer configuration).

## 3. Who have you worked with?

Design and implementation carried out in complete autonomy. The official Prisma, PostgreSQL and Supabase documentation was central, especially for the PgBouncer configuration imposed by Supabase (pgbouncer=true, connection\_limit=1). The project's Prisma community forums and GitHub outputs helped address configuration issues specific to using Prisma and Supabase together.

## 4. Background

|                                                    |   |                                              |                          |
|----------------------------------------------------|---|----------------------------------------------|--------------------------|
| Name of the company, organization or association ▶ |   | Independent training — Vite Gourmand Project |                          |
| Construction site, workshop, service               | ▶ | Full-stack development in autonomy           |                          |
| Fiscal Period                                      | ▶ | From                                         | 01/01/2026 to 15/02/2026 |

## 5. Additional information (optional)

Two Supabase URLs are used: PgBouncer (port 6543) for the application, direct connection (port 5432) for migrations. GDPR anonymization is carried out as an atomic transaction: personal data replaced, sessions deleted, orders kept for accounting obligations.

## Typical activity 2

Develop the back-end part of a secure web or mobile web application

Example 2 ▶ *Develop SQL and NoSQL data access components*

### 1. Describe the tasks or operations you performed, and under what conditions:

I designed and implemented all the components that allow the application to read, write and manipulate the data. The architecture is based on two complementary storage systems: a PostgreSQL relational database for transactional and business data, and a MongoDB database for analytical data, audit logs, and denormalized snapshots.

#### Architecture analysis and technical choices

Before developing the components, I thought about the division of responsibilities between the two systems. PostgreSQL, thanks to its integrity constraints and ACID transactions, is suitable for all data requiring strict consistency: users, commands, menus, discounts, fidelity, notices. MongoDB, thanks to its schema flexibility and write performance, is suitable for data that can vary in schema and that does not require large dependencies of data models: dashboard statistics, activity logs, denormalized snapshots for reports.

#### Prisma Connection Service (SQL)

The first component developed is the PostgreSQL connection service. It extends the Prisma client class and integrates NestJS lifecycle hooks: connection established at module startup, close specific to module destruction with release of the connection pool. The connection uses the pg driver from Node.js with a connection pool. In development mode, all generated SQL queries are saved in the console. This service is declared as a global provider in NestJS, available by dependency injection in all modules without reimporting.

#### Standardisation de la pagination

To ensure consistency across all playback components, I defined a shared utility function for pagination. All services returning lists run list and count queries in parallel with Promise.all, and format the result as an object containing the items and a metadata block (current page, size, total, number of pages).

#### MongoDB Connection Client (NoSQL)

For MongoDB, I developed a connection client implementing the Singleton pattern. It connects to the MongoDB Atlas cluster with the server API in strict mode, a pool of up to ten concurrent connections, a minimum connection that is maintained at all times, and a thirty-second idle timeout. Login, logout, and ping functions are exposed for lifecycle management.

# PROFESSIONAL RECORD (PD)

## Securing components

Access to data is protected on several levels. I use a global validation pipe inspecting each request (here, we use class-validator, a nestjs library), if the validation rules reject undeclared properties it will list every invalid field. A JWT Passport policy extracts and verifies the token on each request. I also use a NestJS guard that checks the role on protected roads (see Fig. 4 and Fig. 6).

## Testing and verification

To verify that the components were working properly, I used a Postman collection covering a selection of API endpoints, organized into end-to-end scenarios: sign-up, login, checkout journey, discount management, loyalty, and unauthorized access checking.

## 2. Specify the means used:

**NestJS:** A back-end framework that structures modules, services, controllers, guards, and man-in-the-middle.

**Prisma ORM:** Schema modeling, generation of the typed TypeScript client, execution of SQL queries.

**@prisma/adapters-pg:** Prisma native adapter for PostgreSQL with connection pool management.

**PostgreSQL / Supabase:** Cloud relational database (PgBouncer transactional pooler for the application, session pooler for migrations).

**MongoDB Atlas:** Cloud NoSQL database, native driver Node.js (without Mongoose), pool of ten connections.

**class-validator + class-transformer:** Validation and transformation of incoming data via TypeScript decorators on DTOs.

**Passport.js + @nestjs/jwt:** Authentication by JWT policy, extraction from the Authorization: Bearer header.

**bcrypt:** Hash passwords (12 rounds of salting).

**@nestjs/throttler:** Throttling the request rate (20 per second, 100 per ten seconds, 300 per minute).

**Swagger / OpenAPI:** Automatically generated interactive documentation used to test endpoints.

**Postman:** A collection of 71 automated tests covering all API use cases.

**Docker:** Containerization that ensures the repeatability of the runtime environment.

## 3. Who have you worked with?

This project was developed entirely autonomously.

## 4. Background

Name of the company, organization or association ▶

Independent training — Vite Gourmand Project

# PROFESSIONAL RECORD (PD)

Construction site,  
workshop, service

Fiscal Period

► *Full-stack development in autonomy*

► From 01/01/2026 to 15/02/2026

## 5. Additional information (optional)

## Typical activity 2

Example 3 ▶

Develop the back-end part of a secure web or mobile web application

Develop server-side business components and document the deployment of a dynamic web or mobile web application

### 1. Describe the tasks or operations you performed, and under what conditions:

*As part of the Vite Gourmand project, I developed all the components that make up the server-side business logic. All this with a modular architecture that is able to evolve without calling into question the existing components.*

#### **NestJS modular architecture**

*I mapped the business areas and defined a NestJS module for each one (see Fig. 10). Each module encapsulates its controller, service, and DTOs (Data Transfer Object = a structure describing the shape of data sent or received between API -> controller -> services) layers, and exposes to the outside only what is needed by the other modules.*

#### **Setting up the entry point**

*Here, the principle of defense in depth, often called **defense in depth**, is used. The main idea of this model is that no single layer protects against everything. But if an attacker breaks through the first layers, he falls on the second. Each layer has its own difficulty in being pierced.*

*Thus, before accepting the first request, the server traverses a security assembly line. Each step has a specific role.*

*We force HTTPS, any request that arrives in HTTP is immediately redirected in 308 to HTTPS. There is no negotiation possible.*

*As for the size of the request bodies, they are capped at 10 MB. It is a simple protection to add against giant requests that would saturate the server's memory.*

*Helmet lays the foundation for safety headers. It's about twenty headers in one line, with a ban on embedding the site in an iframe thanks to the attribute (X-Frame-Options: DENY).*

*We protect it against MIME-type **sniffing**, strict referent policy, HSTS of one year in production. The configuration is slightly adjusted to allow the Google OAuth login popup window – otherwise the "Sign in with Google" button would be blocked by the browser.*

*Permissions-Policy removes access to sensors. The server explicitly states that the app does not use the camera, microphone, or geolocation. If a malicious script tries to access it, the browser refuses. We have also activated compression. JSON responses are gzipped before being sent. This allows pages to load faster, bandwidth is reduced.*

*The CORS only opens the door to known origins. Only the front-end URL configured in the environment variables can send cross-origin requests. Session cookies are allowed in these exchanges, which are essential for the HttpOnly JWT to travel automatically. We also had to protect ourselves against the*

*flaws linked to cookies. This is what CSRF protection allows. An HttpOnly cookie cannot be read by JavaScript, but an attacker can still forge a request that embeds it. That's why the double-cookie pattern was used. Indeed, the front must read a second readable cookie (vgcsrftoken) and return it as a header. Without it, any mutation is rejected with a 403.*

Protection has also been put on the /api prefix which isolates backend routes. All NestJS routes are automatically prefixed. For example, /api/orders, /api/auth, etc. Routes without this prefix point to the React front end in production.

As for the documentation, it is generated at startup. Each time the server is launched, NestJS goes through all the decorators '@ApiTags', '@ApiOperation', '@ApiProperty' and builds an interactive interface accessible on /api/docs with no files to maintain by hand.

## **Data Transfer Objects (DTOs)**

*Each API route rejects so-called malformed data before it even reaches the business code. This is the role of DTOs; objects that describe exactly what a road accepts and reject everything else. For example. The DTO checks six things in order*

- 1. Check l'email*
- 2. That the password policy is respected*
- 3. That the first name is between 2 and 200 characters.*
- 4. That the "I agree" box has been explicitly checked. If it is not. The API will deny the request with a clear error message.*

*For password reset, the rules are stricter: a regular expression verifies that the new password complies with the password policy. No weak passwords are accepted for a sensitive operation.*

## **Interceptors**

*First of all, we must define an interceptor, not to be confused with exceptions. The interceptor executes before and after the handler, when everything goes well. TransformInterceptor is the best example of this. It will receive the request, runs the return handler next.handle().pipe()*

*Afterward, it receives the response and wraps it in a uniform JSON structure.*

*The transform interceptor wraps each successful response in a uniform JSON structure*

```
{
  success:false,
  statusCode: number,
  message: string,
  error: string,
  path: string
  timestamp: string,
  details?: unknown
}
```

*The log interceptor: it measures the duration of each request and records the values of the attributes of the JS page. The HTTP interceptor feeds the service with in-memory logs that keep the last five hundred entries for the administration interface.*



## Exception Filters

Two filters ensure that all errors are returned in the standard format. The HTTP filter captures NestJS exceptions and extracts message, code, and validation details. The global filter captures all unhandled exceptions; in development it exposes the actual message, in production it returns a generic message that does not leak any information about the internal implementation.

## Validation Pipes

When a request arrives, two filters are applied:

The first checks the shape of the data.

For example, if a required field is missing, if an email is invalid, if a password is too short, all of this will be rejected immediately. Which is useful: all the errors are listed at once. If you send a form with an invalid MDP and invalid email. All of this will be rejected immediately. Fields not declared in the dto are also rejected. Unable to inject unplanned data.

The parsing pipe, on the other hand, will check that the value is an integer in the PostgreSQL MININT range  $\leq x \leq \text{MAX\_INT}$  thus preventing overflow errors at the database level.

The second filter for numeric identifiers in URLs - `/api/orders/42`, `/api/users/7`. PostgreSQL stores these IDs as 32-bit signed integers: if someone sends `/api/orders/9999999999`, the request exceeds the capacity of the type and crashes in the database. Thanks to this pipe, we check that the value is indeed an integer within the allowed range before going any further.

## Order status machine

The life cycle of an order follows a state machine: awaiting confirmation, confirmed, in preparation, ready, being delivered, delivered, or canceled. Only certain transitions are allowed. Changes (address, schedule, special instructions) and cancellation are only possible as long as the status is pending or confirmed. Each status change is plotted in a history table with the old status, the new status, any notes, and the timestamp. A nuance should be made: via the admin route `'POST:id/status'`, an employee can also cancel from preparing since the restriction only applies to the `/cancel` route exposed to the client.

## Email Sending Service

The email service is connected to a Titan Email SMTP server on port 465 in implicit TLS, with connection verification at startup. Each send produces a plain text fallback from HTML for compatibility with non-HTML clients. Sends are asynchronous and non-blocking for auth flows and unlike newsletter sends which are awaited — it's intentional. the API responds without waiting for the submission confirmation.

## Query Rate Throttling

A rate-limiting module is configured with three levels: twenty requests per second against sudden peaks, one hundred per ten seconds for moderate bursts, three hundred per minute for the long window. These limits are automatically disabled in the test environment.

## Deployment of the web page in fly.io

We hadn't talked about it until then, fly.io was chosen instead of others as a vercel simply because it was

*the first to fall into my hands. In any case, I had to learn one or the other. There is no purely technical choice behind his selection. Fly.io is part of what is known as a PAAS that deploys Docker containers directly on bare-metal machines. The advantage is that you don't have a server to manage, automatic HTTPS thanks to the Let's Encrypt protocol.*

*The build is done in 3 steps in a single Dockerfile:*

- 1. npm run build for the front end*
- 2. Backend prisma generate + npm run build*
- 3. In runtime, the final image is lightened using ~alpine*

*The final image contains only the production dependencies. The compiled frontend is copied into '/app/public' and served directly by NestJS – one container, one port (8080).*

*To connect API requests from the front, instead of using a API\_URL, we purposely use relative requests /api/... — so the frontend doesn't need to know the domain.*

*The configuration was ultimately tedious.*

*We configured force\_https=true with Fly, which automatically redirects HTTP to HTTPS. Advantage: no turnover.*

*Every 15 seconds, the status of the web server is checked via requests such as GET /api.*

*VM : 1 shared CPU, 1 GB RAM.*

*And we stored the environment secrets in fly.io with the flyctl secrets set command.*

*The image runs with a non-root user nestjs (uid 1001) and tini is used in PID 1 to properly handle UNIX signals (SIGTERM → clean shutdown of the Node process).*

*Finally, flyctl is encapsulated in a Docker Compose service so that it doesn't have to be installed on the host machine. (see Fig.21).*

## **Testing and Documentation**

*The API is documented through Swagger and tested with a Postman collection of seventy-one tests covering nominal traversals and error cases, including verifying that protected routes reject unauthenticated or insufficiently authorized requests. I use JSDoc to comment out functions.*

## **2. Specify the means used:**

# PROFESSIONAL RECORD (PD)

**NestJS** : Back-end framework structuring modules, controllers, services, guards, interceptors, filters, and pipes.

**TypeScript**: Strict typing of all components (DTOs, JWT payloads, extended Express requests).

**class-validator + class-transformer**: Validation and transformation decorators on DTOs.

**Passport.js + @nestjs/passport**: JWT and Google OAuth 2.0 authentication policies.

**@nestjs/jwt**: Generation and verification of JWT tokens (15-minute access, 1-hour reset).

**bcrypt**: Hashing passwords with 12 rounds of salting, secure verification.

**@nestjs/throttler**: Limitation of the request rate to three levels (short, medium, long).

**@nestjs/cache-manager**: Global in-memory cache, TTL 60 seconds, maximum 100 entries.

**Nodemailer** : Envoi d'emails via SMTP Titan Email (port 465, TLS implicite).

**Helmet**: Security HTTP headers, configuration suitable for Google OAuth popup.

**Compression**: Compression of HTTP responses to reduce bandwidth.

**Swagger / OpenAPI**: Interactive documentation generated from NestJS decorators.

**Postman**: A collection of 71 automated tests covering all endpoints.

**Docker**: Containerization that ensures the reproducibility of the environment.

## 3. Who have you worked with?

All server-side business components have been developed autonomously. This posture led me to simultaneously take charge of the definition of the architecture (choice of NestJS patterns, organization of modules, security strategy), the development of components, and the validation of their behavior by testing. Without a team to challenge decisions, I used official NestJS documentation and community best practice guides to validate my architectural choices—including configuring global guards, standardized error handling, and implementing multi-policy authentication.

## 4. Background

| Name of the company, organization or association ▶ |   | Independent training — Vite Gourmand Project |
|----------------------------------------------------|---|----------------------------------------------|
| Construction site, workshop, service               | ▶ | Full-stack development in autonomy           |
| Fiscal Period                                      | ▶ | From 01/01/2026 to 15/02/2026                |

## 5. Additional information (optional)

## PROFESSIONAL RECORD (PD)

*Each request traverses an ordered chain: rate limiting → JWT → role → DTO validation → business service. This architecture is extensible without modifying existing controllers. GDPR compliance is imposed at the DTO level: consent must be exactly true, with no possibility of circumvention.*

## PROFESSIONAL RECORD (PD)

### Titles, diplomas, CQP, training certificates

| Title                                                                                 | Authority or body                  | Date                         |
|---------------------------------------------------------------------------------------|------------------------------------|------------------------------|
| Scientific Baccalaureate                                                              | Reims Academy – National Education | 01/07/2017                   |
| Certificate of Academic Results – Common Core Curriculum (level 4.62 / 21), 42 Madrid | 42 Madrid Fundación Telefónica     | 25/05/2026                   |
| Click here.                                                                           | Click here to type text.           | Click here to select a date. |

## PROFESSIONAL RECORD (PD)

### Declaration of honour

I, the undersigned, .....*LESIEUR.DYLAN*.....,  
declares on my honour that the information provided in this file is accurate and that I am the  
author of the attached achievements.

Done at *Madrid* ..... The *21/05/2026* .....

to assert what is right.

Signature :

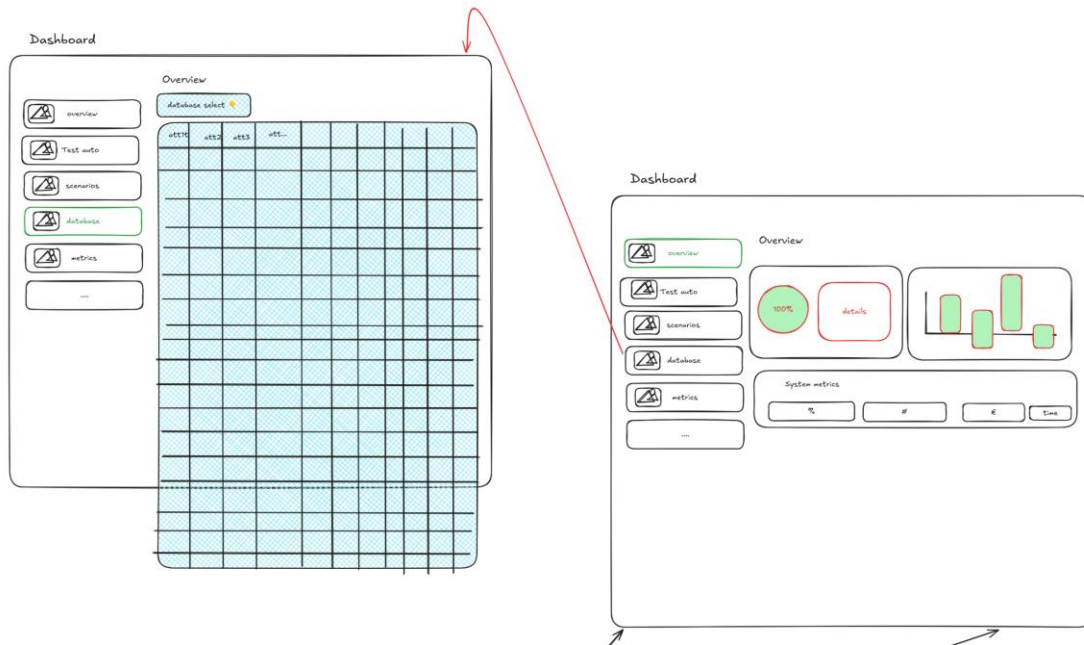


## Documents illustrating professional practice

*(optional)*

| Title                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fig. 1 — Low-fidelity mock-up -- DevBoard admin interface, an admin, employee, customer space                                                                                              |
| Fig. 2 — Low-fidelity mockup -- client-side interface with 3 pages, home, menus, contact                                                                                                   |
| Fig. 3 — Loyalty and marketing database relationship                                                                                                                                       |
| Fig. 4 — Conceptual diagram of the relational database for identity, role, security                                                                                                        |
| Fig. 5 — Entity diagram menu to ingredient relationships                                                                                                                                   |
| Fig. 6 — Actor matrix × use cases (rights per role)                                                                                                                                        |
| Fig. 7 — Sequence diagram of an order take.                                                                                                                                                |
| Fig. 8 — container running through the makefile target that orchestrates docker-compose to quickly free up memory and activate the program in its own isolated environment.                |
| Fig. 9 — Detail of the container orchestration being developed                                                                                                                             |
| Fig. 10 — NestJS request lifecycle: Layered architecture illustrating the top-down processing of the incoming HTTP request until the response returned to the client                       |
| Fig. 11 — QA tester sonarcloud overview of the project context as of May 26, 2025                                                                                                          |
| Fig. 12 — lighthouse test of the real website of the home page                                                                                                                             |
| Fig. 13 — lighthouse test of the actual website of the menu page                                                                                                                           |
| Fig. 14 — Pipeline CI from github Actions with verified and functional tests                                                                                                               |
| Fig. 15 — Mindmap map of the application architecture and underlying specifications                                                                                                        |
| Fig. 16 — Extracts from the css tokens used on the front-end side                                                                                                                          |
| Fig. 17 — New docker architectural layer of microservices                                                                                                                                  |
| Fig. 18 — excerpts from the text robot that allows you to communicate directly with the browsers' robots                                                                                   |
| Fig. 19 — code snippet that shows the algorithm and usage of bcrypt which is salted 12 times                                                                                               |
| Fig. 20 — Proof of aggregation of the pgbouncer property to connect with supabase because they use a strict connection simultaneously. Without pooling, the server would quickly saturate. |
| Fig. 21 — excerpt from my makefile orchestration code that manages fly.io                                                                                                                  |

# PROFESSIONAL RECORD (PD)

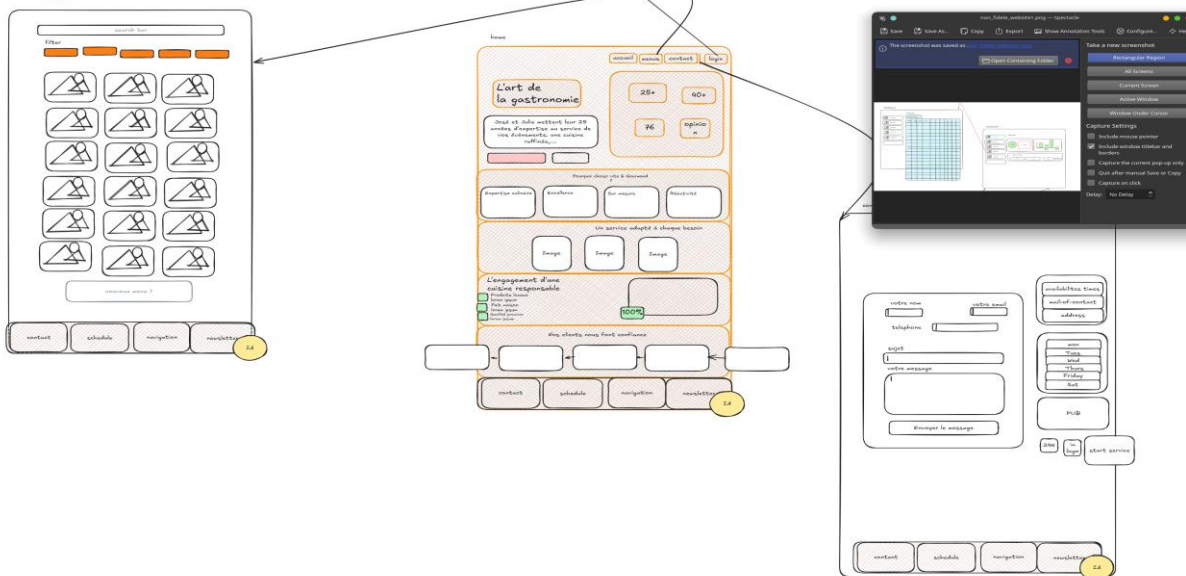




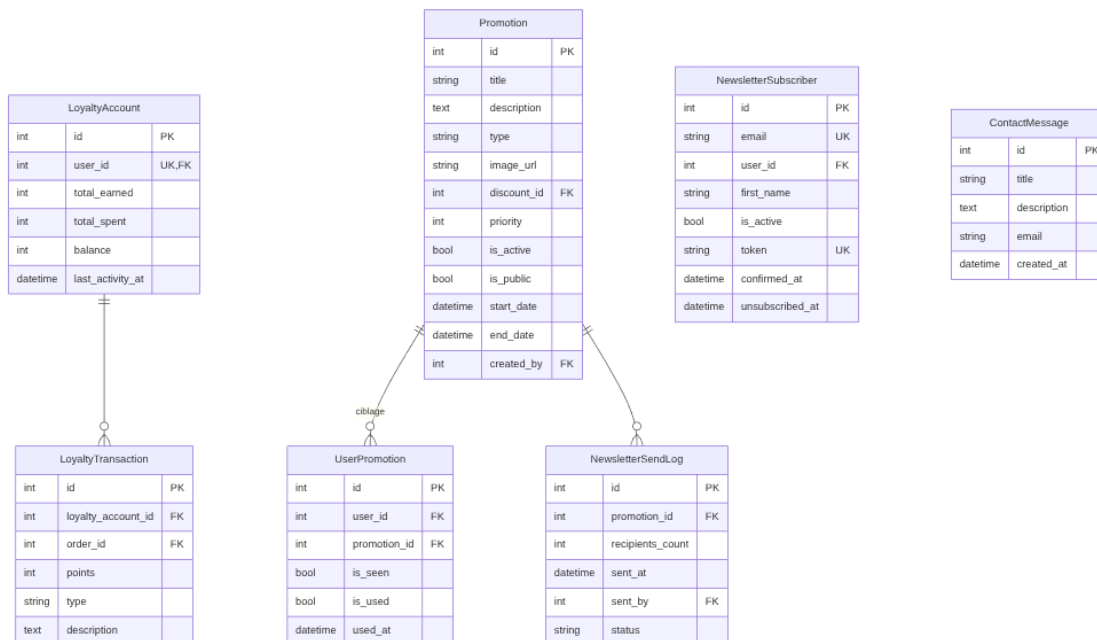
# PROFESSIONAL RECORD (PD)

## APPENDICES

(If the RC provides for it)

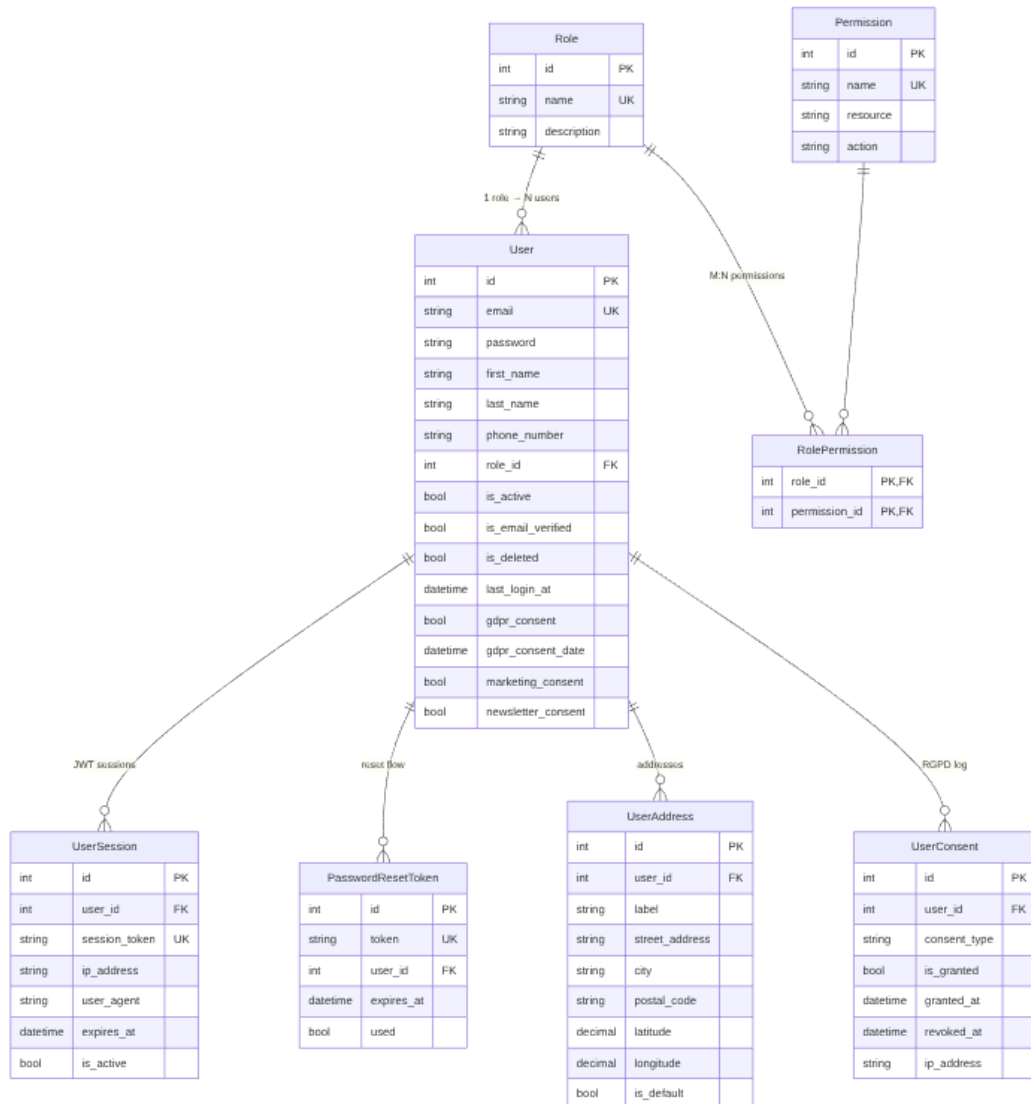


### 2.4 Fidélité & marketing

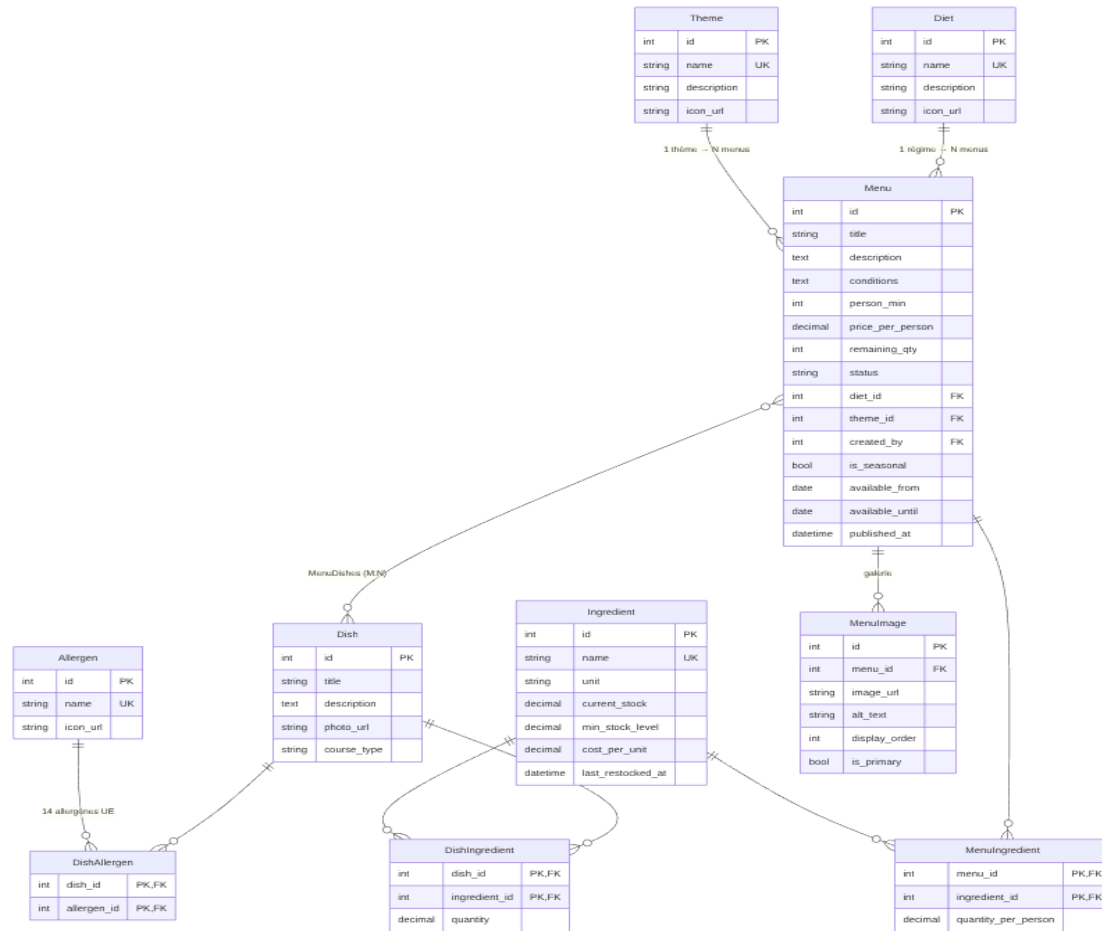


## 2. Vue par domaine

### 2.1 Identité, sessions, sécurité

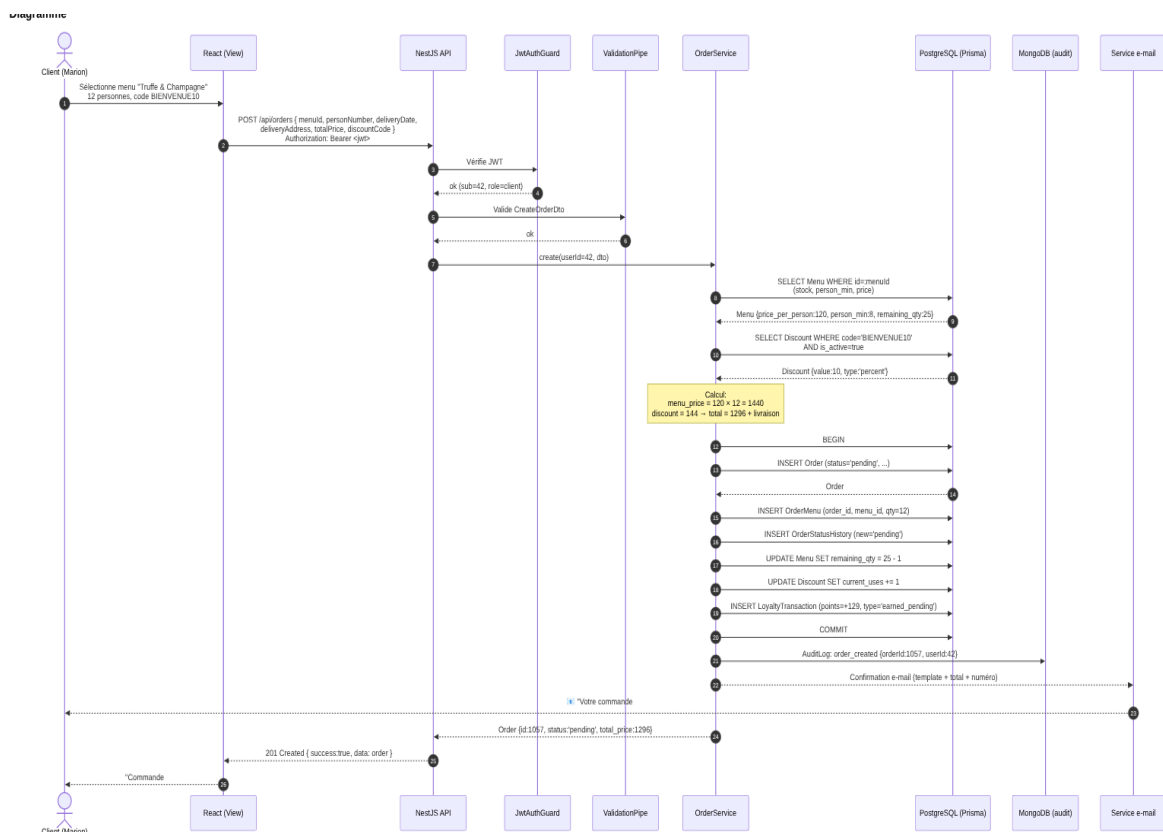


## 2.2 Catalogue : menus, plats, allergènes, ingrédients



## 4. Matrice acteur × cas d'utilisation (extrait)

| Cas d'utilisation          | Visiteur | Client | Employé | Admin |
|----------------------------|----------|--------|---------|-------|
| Consulter menus            | ✓        | ✓      | ✓       | ✓     |
| Envoyer formulaire contact | ✓        | ✓      | —       | —     |
| Créer un compte            | ✓        | —      | —       | —     |
| Passer une commande        | —        | ✓      | —       | —     |
| Annuler avant confirmation | —        | ✓      | —       | —     |
| Modifier statut commande   | —        | —      | ✓       | ✓     |
| Annuler commande confirmée | —        | —      | ✓       | ✓     |
| Modérer un avis            | —        | —      | —       | ✓     |
| Créer un menu              | —        | —      | —       | ✓     |
| Créer un employé           | —        | —      | —       | ✓     |
| Lancer une newsletter      | —        | —      | —       | ✓     |
| Consulter analytics        | —        | —      | —       | ✓     |
| Demander suppression RGPD  | —        | ✓      | —       | —     |
| Approuver suppression RGPD | —        | —      | —       | ✓     |



#### Points techniques

**dlesieur@dlesieur:~/Documents/vite-gourmand\$ make turn-off**

**SHUTTING DOWN VITE GOURMAND**

- Stopping frontend...  
✓ Frontend stopped
- Stopping backend...  
✓ Backend stopped
- ✓ Everything shut down.

● **dlesieur@dlesieur:~/Documents/vite-gourmand\$ make turn-on**

**TURNING ON VITE GOURMAND**

- 🔧 Starting Backend (NestJS) on port 3000...  
PID: 8929
- 💻 Starting Frontend (Vite) on port 5173...  
PID: 8936
- ✓ Servers started!
- 💻 Frontend → http://localhost:5173
- 🔧 Backend → http://localhost:3000/api

**dlesieur@dlesieur:~/Documents/vite-gourmand\$**

```
d@esieur@d@esieur:~/Documents/vite-gourmand$ make all

VITE GOURMAND - Containerized Development

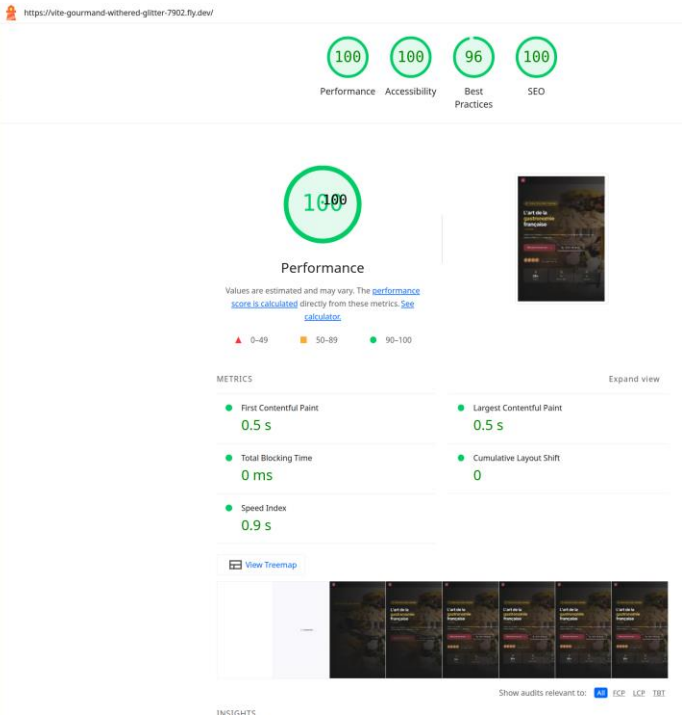
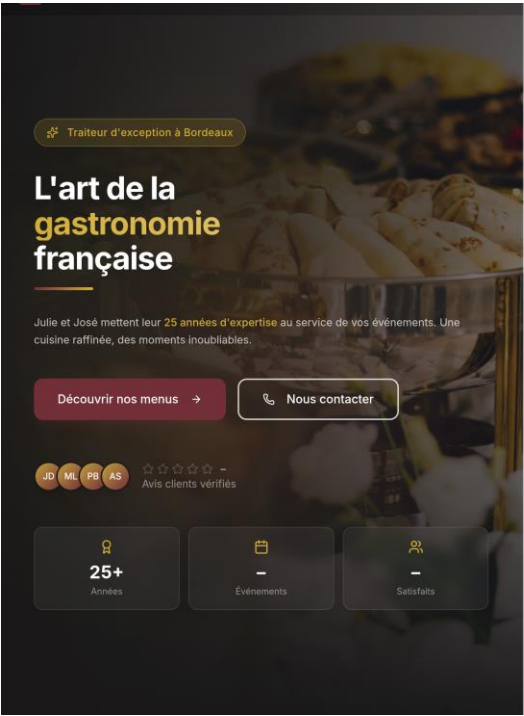
Everything runs inside Docker containers.
Your host machine only needs Docker installed!

Steps:
1. 🚀 Build dev container (Node.js 22 Alpine)
2. 📁 Fetch .env from Bitwarden (interactive)
3. 📦 Install dependencies inside container
4. 🏗️ Generate Prisma client & compile TypeScript
5. 🏃 Start dev servers inside container

Building Development Container

Service dev Building
Sending build context to Docker daemon 60.09MB
Step 1/11 : FROM node:22-alpine
--> 968df39aede
Step 2/11 : RUN apk add --no-cache bash curl git jq make psmisc procps python3 g++ openssl libc6-compat
--> Running in b672c5adceb
( 1/53) Installing ncurses-terminfo-base (6.5_p20251123-r0)
( 2/53) Installing libncursesw (6.5_p20251123-r0)
( 3/53) Installing readline (8.3.1-r0)
( 4/53) Installing bash (5.3.3-r1)
Executing bash-5.3.3-r1.post-install
( 5/53) Installing brotli-libs (1.2.0-r0)
( 6/53) Installing c-ares (1.34.6-r0)
( 7/53) Installing libunistring (1.4.1-r0)
( 8/53) Installing libidn2 (2.3.8-r0)
( 9/53) Installing nghttp2-libs (1.69.0-r0)
(10/53) Installing libpsl (0.21.5-r3)
(11/53) Installing zstd-libs (1.5.7-r2)
(12/53) Installing libcurl (8.19.0-r0)
(13/53) Installing curl (8.19.0-r0)
(14/53) Installing libstdc++-dev (15.2.0-r2)
(15/53) Installing jansson (2.14.1-r0)
(16/53) Installing binutils (2.45.1-r0)
(17/53) Installing libgomp (15.2.0-r2)
(18/53) Installing libatomic (15.2.0-r2)
(19/53) Installing gmp (6.3.0-r4)
(20/53) Installing isl26 (0.26-r1)
(21/53) Installing mpfr4 (4.2.2-r0)
(22/53) Installing mpcl (1.3.1-r1)
(23/53) Installing gcc (15.2.0-r2)
(24/53) Installing musl-dev (1.2.5-r23)
(25/53) Installing g++ (15.2.0-r2)
(26/53) Installing libevent (2.7.5-r0)
(27/53) Installing pcre2 (10.47-r0)
(28/53) Installing git (2.52.0-r0)
(29/53) Installing git-init-template (2.52.0-r0)
(30/53) Installing oniguruma (6.9.10-r0)
(31/53) Installing jq (1.8.1-r0)
(32/53) Installing musl-obstack (1.2.3-r2)
(33/53) Installing libucontext (1.3.3-r0)
(34/53) Installing gcompat (1.1.0-r4)
(35/53) Installing make (4.4.1-r3)
(36/53) Installing openssl (3.5.6-r0)
(37/53) Installing libintl (0.24.1-r1)
(38/53) Installing skelibs-libs (2.14.4.0-r0)
(39/53) Installing utmps-libs (0.1.3.1-r0)
(40/53) Installing libproc2 (4.0.5-r0)
(41/53) Installing procps-ng (4.0.5-r0)
(42/53) Installing psmisc (23.7-r0)
(43/53) Installing libbz2 (1.0.8-r6)
(44/53) Installing libffi (3.5.2-r0)
```





🏠

NOTRE CARTE

# Nos Menus

Des créations culinaires d'exception, pensées pour sublimer chacun de vos événements. Filtrez par thème, régime alimentaire ou budget.

Rechercher un menu, un thème...

Filtres

Tous

Anniversaire

Baptême

Barbecue

Brunch

Cocktail

Entreprise

6 menus disponibles

🍷 Mariage

### Menu Végétarien Élégant

végétarien

3 allergies

Alternative végétarienne raffinée

Min. 20 pers. 65 €/pers.

🍷 Entreprise

### Cocktail Entreprise

classique

5 allergies

Assortiment de canapés et mignardises

Min. 30 pers. 45 €/pers.

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

← CI Pipeline

update #21

Summary

All jobs

1 - Lint (ESLint)

2 - Unit tests (Jest)

3 - E2E tests (Jest + DB)

4 - Custom validation tests

5 - Postman API tests

6 - Test summary (all suites)

7 - Frontend security

8 - Build (NestJS + Vite)

9 - SonarCloud

Run details

Usage

Workflow file

Triggered via push 6 minutes ago

Status

Total duration

Artifacts

LEsdyran pushed

Success

6m 30s

8

ci-cd.yml

on: push

1 - Lint (ESLint)

2 - Unit tests (Jest)

3 - E2E tests (Jest + DB)

4 - Custom validation tests

5 - Postman API tests

6 - Test summary (all suites)

7 - Frontend security

8 - Build (NestJS + Vite)

9 - SonarCloud

6 - Test summary (all suites) summary

Test Summary

| Suite             | Passed | Failed | Total |
|-------------------|--------|--------|-------|
| Jest unit         | 324    | 0      | 324   |
| Jest e2e          | 346    | 0      | 346   |
| Custom validation | 153    | 0      | 153   |
| Postman API       | 71     | 0      | 71    |
| Total             | 894    | 0      | 894   |

https://vite.gourmand.withered-glitter-7902.fly.dev/menus

91

Performance

98

Accessibility

92

Best Practices

100

SEO

91

Performance

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculation.](#)

0-49

50-89

90-100

METRICS

Expand view

First Contentful Paint

2.2 s

Largest Contentful Paint

2.3 s

Total Blocking Time

0 ms

Cumulative Layout Shift

0

Speed Index

6.0 s

View Treemap

Show audits relevant to:

FCP

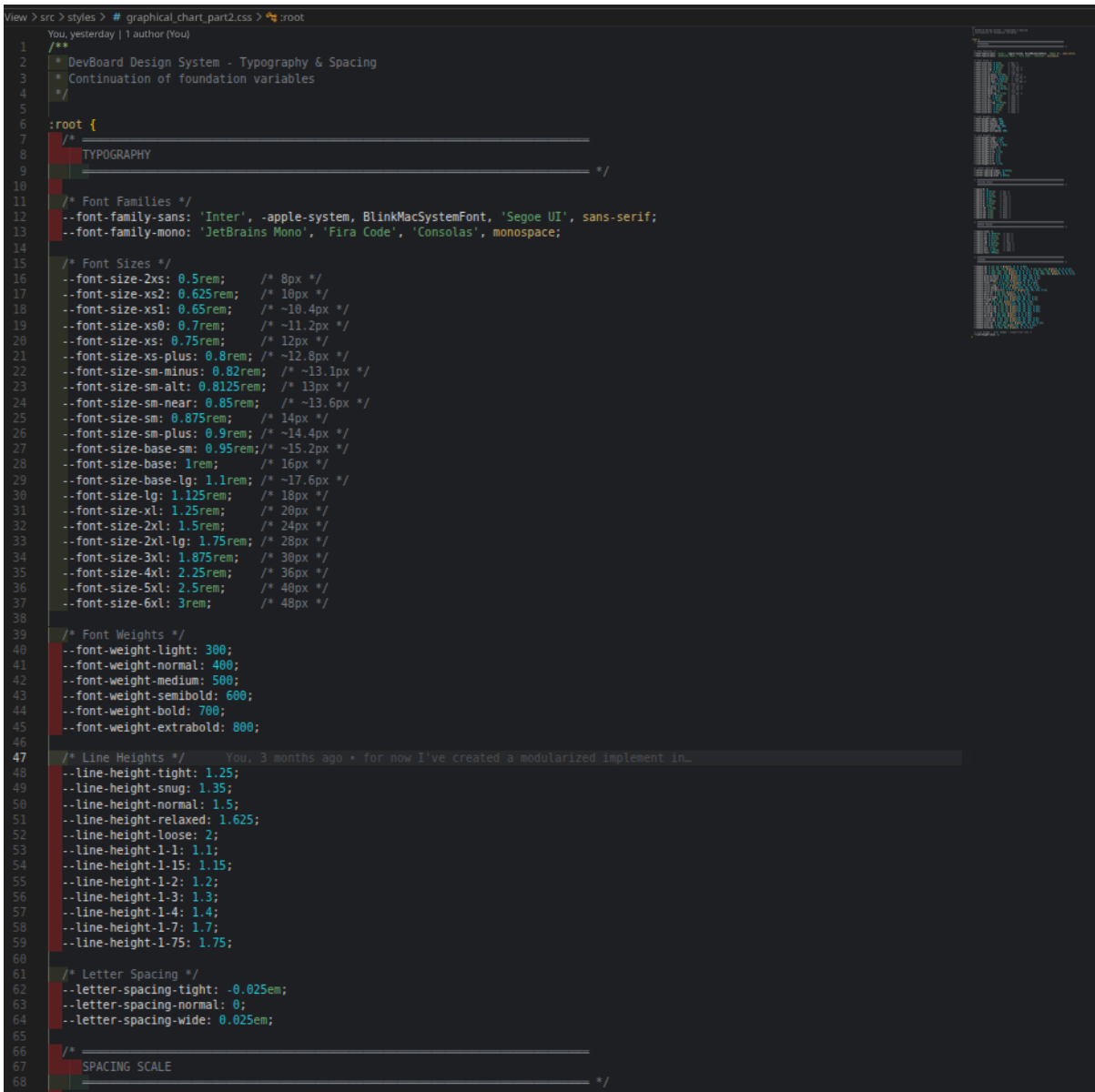
LCP

TBT

PROFESSIONAL FILE-Version of 01/06/2016

Page 40





```
dlesieur ~/.../vite-gourmand/infrastructure main --- 01:07:20
> tree -d
.
├── contracts
├── services
│   ├── app
│   │   ├── config
│   │   └── scripts
│   ├── dev
│   │   ├── config
│   │   └── scripts
│   ├── fly
│   │   ├── config
│   │   └── scripts
│   ├── mongo
│   │   ├── config
│   │   └── scripts
│   ├── postgres
│   │   ├── config
│   │   └── scripts
│   ├── secrets
│   │   ├── config
│   │   └── scripts
└── sonarcloud
    └── results

23 directories
```

```
100% yesterday | 1 author (100%)
User-agent: *
Allow: /
Disallow: /admin
Disallow: /dashboard
Disallow: /employee

Sitemap: https://vite-gourmand.fr/sitemap.xml
```

```

import { Injectable, UnauthorizedException } from '@nestjs/common';
import * as bcrypt from 'bcrypt';

const SALT_ROUNDS = 12;

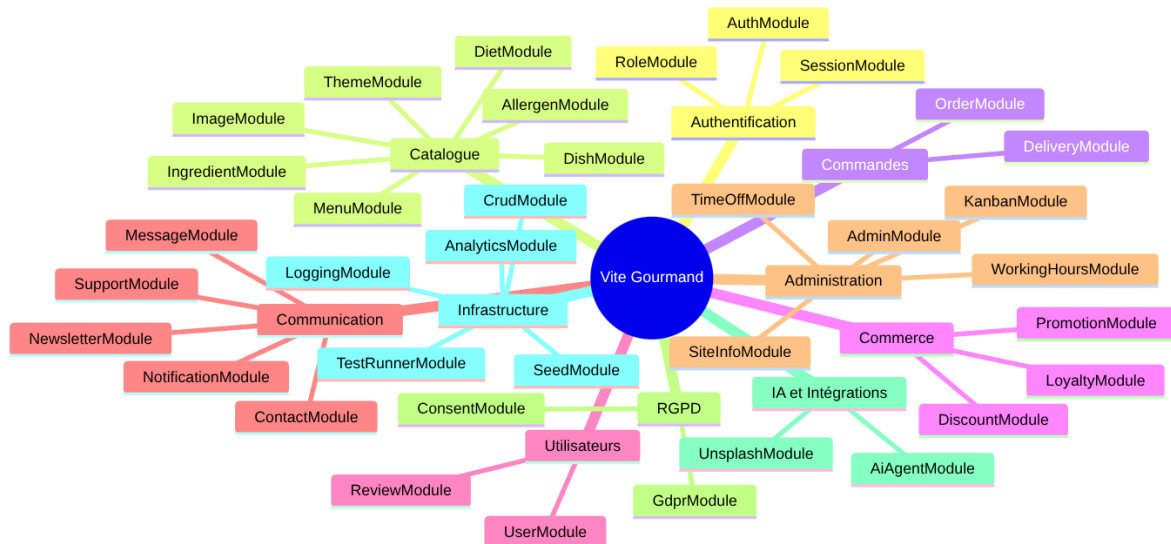
You, yesterday | 1 author (You)
@Injectable()
export class PasswordService {
  /**
   * Hash a plain text password
   */
  async hash(password: string): Promise<string> {
    return bcrypt.hash(password, SALT_ROUNDS);
  }

  /**
   * Verify password against hash
   * @throws UnauthorizedException if password doesn't match
   */
  async verify(password: string, hash: string): Promise<void> {
    const isValid = await bcrypt.compare(password, hash);
    if (!isValid) {
      throw new UnauthorizedException('Invalid credentials');
    }
  }
}

```

You, 3 months ago • new modules created based on legacy code with n...

## 1. Vue d'ensemble — 10 domaines



```

90     ThrottlerModule.forRoot({
91       throttlers:
92         process.env.NODE_ENV === 'test'
93           ? []
94           : [
95             {
96               name: 'short',
97               ttl: 1000, // 1 second
98               limit: 20, // 20 requests per second (SPA fires parallel calls on load)
99             },
100             {
101               name: 'medium',
102               ttl: 10000, // 10 seconds
103               limit: 100, // 100 requests per 10 seconds
104             },
105             {
106               name: 'long',
107               ttl: 60000, // 1 minute
108               limit: 300, // 300 requests per minute
109             },
110           ],
111     }),

```

```

# Construct connection URLs
# Using Transaction pooler (port 6543) for application connections
DATABASE_URL="postgresql://postgres.${SUPABASE_PROJECT_REF}:${ENCODED_PASSWORD}
@aws-0-eu-west-3.pooler.supabase.com:6543/postgres?pgbouncer=true"

```

```

1  ##@ Deployment
2
3  .PHONY: deploy-fly deploy-check deploy-status deploy-logs deploy-certs deploy-fly-shell deploy-safe
4
5  deploy-fly: ## Deploy backend to Fly.io
6      @$(SCRIPTS_PATH)/deploy/fly.sh
7
8  deploy-check: ## Run pre-deployment checks (compile, tests, env)
9      @$(SCRIPTS_PATH)/deploy/check.sh
10
11  deploy-status: ## Show current Fly.io application status
12      @$(SCRIPTS_PATH)/deploy/status.sh
13
14  deploy-logs: ## Stream live Fly.io application logs
15      @$(SCRIPTS_PATH)/deploy/logs.sh
16
17  deploy-certs: ## Inspect/request Fly managed certificates (CREATE_CERTS=true to create)
18      @CREATE_CERTS="$(CREATE_CERTS)" HOSTS="$(HOSTS)" $(SCRIPTS_PATH)/deploy/fly-certificates.sh
19
20  deploy-fly-shell: ## Open a shell in the Dockerized Fly CLI service
21      @if [ -f .env.production ]; then \
22          $(DOCKER_COMPOSE) --env-file .env.production --profile tools run --rm fly; \
23      else \
24          $(DOCKER_COMPOSE) --profile tools run --rm fly; \
25      fi
26
27  deploy-safe: deploy-check deploy-fly ## Run pre-checks then deploy to Fly.io
28

```